Beta

Registers, Global, and Local Memory

Last updated on 2024-03-12 | Edit this page

OVERVIEW

Questions

- “What are registers?”
- “How to share data between host and GPU?”
- “Which memory is accessible to threads and thread blocks?”

Objectives

- “Understanding the difference between registers and device memory”
- “Understanding the difference between local and global memory”

Now that we know how to write a CUDA kernel to run code on the GPU, and how to use the Python interface provided by CuPy to execute it, it is time to look at the different memory spaces in the CUDA programming model.

Registers

Registers are fast on-chip memories that are used to store operands for the operations executed by the computing cores.

Did we encounter registers in the `vector_add` code used in the previous episode? Yes we did! The variable `item` is, in fact, stored in a register for at least part, if not all, of a thread’s execution. In general all scalar variables defined in CUDA code are stored in registers.

Registers are local to a thread, and each thread has exclusive access to its own registers: values in registers cannot be accessed by other threads, even from the same block, and are not available for the host. Registers are also not permanent, therefore data stored in registers is only available during the execution of a thread.

CHALLENGE: HOW MANY REGISTERS ARE WE USING?

Can you guess how many registers are we using in the following `vector_add` code?

C

```
extern "C"
__global__ void vector_add(const float * A, const float * B, float * C, const int size)
{
   int item = (blockIdx.x * blockDim.x) + threadIdx.x;

   if ( item < size )
   {
      C[item] = A[item] + B[item];
   }
}
```

Solution

In general, it is not possible to exactly know how many registers the compiler will use without examining the output generated by the compiler itself. However, we can roughly estimate the amount of necessary registers based on the variables used. We most probably need one register to store the variable `item`, two registers to store the content of `A[item]` and `B[item]`, and one additional register to store the sum `A[item] + B[item]`. So the number of registers that `vector_add` probably uses is 4.

If we want to make registers use more explicit in the `vector_add` code, we can try to rewrite it in a slightly different, but equivalent, way.

```C
extern "C"
__global__ void vector_add(const float * A, const float * B, float * C, const int size)
{
   int item = (blockIdx.x * blockDim.x) + threadIdx.x;
   float temp_a, temp_b, temp_c;

   if ( item < size )
   {
       temp_a = A[item];
       temp_b = B[item];
       temp_c = temp_a + temp_b;
       C[item] = temp_c;
   }
}
```

In this new version of `vector_add` we explicitly declare three `float` variables to store the values loaded from memory and the sum of our input items, making the estimation of used registers more obvious.

This it totally unnecessary in the case of our example, because the compiler will determine on its own the right amount of registers to allocate per thread, and what to store in them. However, explicit register usage can be important for reusing items already loaded from memory.

> CALLOUT
>
> Registers are the fastest memory on the GPU, so using them to increase data reuse is an important performance optimization. We will look at some examples of manually using registers to improve performance in future episodes.

Small CUDA arrays, which size is known at compile time, will also be allocated in registers by the compiler. We can rewrite the previous version of `vector_add` to work with an array of registers.

```C
extern "C"
__global__ void vector_add(const float * A, const float * B, float * C, const int size)
{
   int item = (blockIdx.x * blockDim.x) + threadIdx.x;
   float temp[3];

   if ( item < size )
   {
       temp[0] = A[item];
       temp[1] = B[item];
       temp[2] = temp[0] + temp[1];
       C[item] = temp[2];
   }
}
```

Once again, this is not something that we would normally do, and it is provided only as an example of how to work with arrays of registers.

Global Memory

Global memory can be considered the main memory space of the GPU in CUDA. It is allocated, and managed, by the host, and it is accessible to both the host and the GPU, and for this reason the global memory space can be used to exchange data between the two. It is the largest memory space available, and therefore it can contain much more data than registers, but it is also slower to access. This memory space does not require any special memory space identifier.

CHALLENGE: IDENTIFY WHEN GLOBAL MEMORY IS USED

Observe the code of the following `vector_add` and identify where global memory is used.

```C
extern "C"
__global__ void vector_add(const float * A, const float * B, float * C, const int size)
{
   int item = (blockIdx.x * blockDim.x) + threadIdx.x;

   if ( item < size )
   {
      C[item] = A[item] + B[item];
   }
}
```

Solution

The vectors `A`, `B`, and `C` are stored in global memory.

Memory allocated on the host, and passed as a parameter to a kernel, is by default allocated in global memory.

Global memory is accessible by all threads, from all thread blocks. This means that a thread can read and write any value in global memory.

CALLOUT

While global memory is visible to all threads, remember that global memory is not coherent, and changes made by one thread block may not be available to other thread blocks during the kernel execution. However, all memory operations are finalized when the kernel terminates.

Local Memory

Memory can also be statically allocated from within a kernel, and according to the CUDA programming model such memory will not be global but *local* memory. Local memory is only visible, and therefore accessible, by the thread allocating it. So all threads executing a kernel will have their own privately allocated local memory.

CHALLENGE: USE LOCAL MEMORY

Modify the following of `vector_add` so that intermediate data products are stored in local memory, and only the final result is saved into global memory.

```C
extern "C"
__global__ void vector_add(const float * A, const float * B, float * C, const int size)
{
   int item = (blockIdx.x * blockDim.x) + threadIdx.x;

   if ( item < size )
   {
      C[item] = A[item] + B[item];
   }
}
```

Hint: have a look at the example using an array of registers, but find a way to use a variable and not a constant for the size.

Solution

We need to pass the size of the local array as a new parameter to the kernel, because if we just specified `3` in the code, the compiler would allocate registers and not local memory.

C

```c
extern "C"
__global__ void vector_add(const float * A, const float * B, float * C, const int size, const int local_memory_size)
{
   int item = (blockIdx.x * blockDim.x) + threadIdx.x;
   float local_memory[local_memory_size];

   if ( item < size )
   {
      local_memory[0] = A[item];
      local_memory[1] = B[item];
      local_memory[2] = local_memory[0] + local_memory[1];
      C[item] = local_memory[2];
   }
}
```

The host code could be modified adding one line and changing the way the kernel is called.

PYTHON

```python
local_memory_size = 3
vector_add_gpu((2, 1, 1), (size // 2, 1, 1), (a_gpu, b_gpu, c_gpu, size, local_memory_size))
```

Local memory is not not a particularly fast memory, and in fact it has similar throughput and latency of global memory, but it is much larger than registers. As an example, local memory is automatically used by the CUDA compiler to store spilled registers, i.e. to temporarily store variables that cannot be kept in registers anymore because there is not enough space in the register file, but that will be used again in the future and so cannot be erased.

KEY POINTS

- “Registers can be used to locally store data and avoid repeated memory operations”
- “Global memory is the main memory space and it is used to share data between host and GPU”
- “Local memory is a particular type of memory that can be used to store data that does not fit in registers and is private to a thread”